

ISKOLAI TUDÁSTÁR

Weboldal projekt-specifikáció

Codex 5.6 Terra fejlesztési alapidokumentum

Dokumentum célja:

Egy személyes, hosszú távon bővíthető, modern és jól használható online tanulási tudástár létrehozása a felhasználó saját iskolai jegyzeteiből.

1. Projekt célja

A projekt egy személyes digitális tudástár létrehozása.

A kiindulási anyag a felhasználó saját iskolai jegyzeteiből készített Markdown-tartalom.

A rendszer célja, hogy:

- a kézzel készített iskolai jegyzetekből rendezett digitális tananyag legyen;
- a tananyag tantárgyak és leckék szerint strukturált legyen;
- a tartalom könnyen olvasható és tanulható legyen;
- a teljes tudástárban lehessen keresni;
- az egyes témák és leckék összekapcsolhatók legyenek;
- a rendszer később több száz lecke kezelésére is alkalmas legyen;
- új tananyag hozzáadása ne igényeljen webfejlesztési munkát;
- a tartalom és a weboldal megjelenése legyen egymástól elválasztva;
- az oldal mobiltelefonon, tableten és számítógépen egyaránt jól működjön.

A weboldal ne klasszikus blog legyen.

A megfelelő koncepció:

személyes digitális tankönyv / tudástár / learning wiki / digital garden.

2. Alapvető fejlesztési elv

A projekt legfontosabb alapelve:

A tartalom legyen elválasztva a prezentációtól.

A tananyag Markdown fájlokban legyen tárolva.

A weboldal ezeket a strukturált tartalmakat jelenítse meg egységes sablonok alapján.

Nem cél több száz külön kézzel elkészített HTML-oldalt fenntartani.

A rendszer logikája:

```
Kézzel írt jegyzet
    ↓
Word dokumentum
    ↓
AI / kézi szerkesztés
    ↓
strukturált Markdown
    ↓
tartalomkönyvtár
    ↓
site generator / build folyamat
    ↓
statikus weboldal
```

3. Technológiai irány

A projekt legyen egyszerű, gyors, karbantartható és lehetőleg statikus.

Kötelező technológiák

- HTML5
- CSS3
- JavaScript
- Bootstrap 5
- Markdown
- reszponzív webdesign

Javasolt további technológiák

A Codex választhat megfelelő statikus site generátort vagy könnyű build rendszert, amennyiben az:

- egyszerűen telepíthető;
- jól támogatja a Markdownot;
- jól kezelhető Codexből;
- statikus HTML/CSS/JS kimenetet tud készíteni;
- nem tesz szükségessé adatbázist vagy külön backend szerveret;
- később könnyen bővíthető.

A konkrét framework kiválasztásakor az egyszerűség és a hosszú távú karbantarthatóság legyen elsődleges szempont.

Kerülendő a szükségtelen technológiai túlkomplikálás.

4. A weboldal általános karaktere

A weboldal vizuális stílusa:

modern + letisztult + akadémiai + tankönyvszerű + digitális könyvtár jellegű.

Inspirációs irány:

- modern egyetemi jegyzetplatform;
- digitális könyvtár;
- Notion rendezettsége;
- modern dokumentációs oldalak;
- igényes online tankönyv.

Nem cél:

- gyerekes megjelenés;
- túlzottan színes felület;
- emoji-központú dizájn;
- túlzott animáció;
- „AI dashboard” jelleg;
- sablonos Bootstrap-demo kinézet.

Az oldalnak legyen saját karaktere.

5. Bootstrap használata

A Bootstrap 5 legyen ténylegesen része a projektnek.

Használható többek között:

- navbar;
- container;
- row / col;
- cards;
- badges;
- buttons;
- breadcrumb;
- alerts;
- accordion;
- dropdown;
- offcanvas;
- modal;
- pagination;
- form elemek;
- responsive utility classok.

A Bootstrap alapstílusait azonban szükség esetén saját CSS egészítse ki.

A végeredmény ne nézzen ki alapértelmezett Bootstrap-oldalnak.

6. Reszponzív működés

A weboldal elsődlegesen responsive legyen.

Támogatandó:

- mobiltelefon;
- tablet;
- laptop;
- asztali monitor;
- nagyobb kijelző.

Mobile-first szemlélet javasolt.

Mobilon:

- legyen egyszerű navigáció;
 - a menü legyen összecukható;
 - a tartalom ne legyen túl széles;
 - a kártyák egymás alá rendeződjenek;
 - a képletek és hosszabb szövegek ne törjenek használhatatlanul;
 - a kereső legyen könnyen elérhető.
-

7. Fő navigáció

A weboldal alapvető navigációja:

Főoldal
Tantárgyak
Leckék
Témák
Keresés

Opcionálisan később:

Kedvencek
Ismétlés
Vizsgafelkészülés
Statisztikák

Ezeket jelenleg nem kell teljes funkcionalitással megvalósítani, de az architektúra legyen később bővíthető.

8. Főoldal

A főoldal legyen tanulási dashboard.

Tartalmazzon:

Hero / fejléc

Rövid bemutatkozó szöveg a tudástárról.

Például:

„Az iskolai tanulmányaim digitális tudástára.”

A szöveg legyen egyszerű és visszafogott.

Statisztikák

Automatikusan számolt adatok:

- tantárgyak száma;
- leckék száma;
- témák száma;
- esetleg legutóbbi frissítés dátuma.

Tantárgyak

A tantárgyak kártyák formájában jelenjenek meg.

Minden kártyán:

- tantárgy neve;
- rövid leírás;
- leckék száma;
- opcionálisan ikon vagy egyszerű vizuális jelölés.

Legutóbbi leckék

A legutóbb hozzáadott vagy legfrissebb leckék listája.

Minden elem:

- dátum;
 - tantárgy;
 - lecke címe;
 - rövid kivonat.
-

9. Tantárgyak oldal

A `/subjects` vagy ennek megfelelő útvonal tartalmazza az összes tantárgyat.

Például:

Tantárgyak

Mechanika, Gépelemek
Elektrotechnika, elektronika, villamosságtan
Gépjármű-szerkezetan
Elektrotan

A tényleges tantárgynevek a forrásanyag alapján legyenek használva.

Ha egy rövidítés vagy tantárgynév nem egyértelmű a forrásanyagból, a rendszer ne találjon ki hozzá nevet.

10. Tantárgyoldal

Minden tantárgynak legyen saját oldala.

Példa:

Elektrotan

Leckék

2026. szeptember 3.
Villamos alapmennyiségek

A tantárgyoldal tartalmazza:

- tantárgy nevét;
- rövid leírását, ha rendelkezésre áll;
- leckék számát;
- leckék listáját;
- dátum szerinti rendezést;
- opcionális témaszűrést.

A leckék alapértelmezett sorrendje legyen időrendi.

11. Lecke mint alapegység

A rendszer alapvető tartalmi egysége egy **lecke**.

Egy lecke legalább az alábbi adatokat tartalmazza:

```
title:
subject:
date:
topics:
content:
```

Javasolt teljes metadata:

```
---
title: "Villamos alapmennyiségek"
subject: "Elektrotan"
date: "2026-09-03"
topics:
  - "Elektromos töltés"
  - "Elektromos mező"
  - "Feszültség"
  - "Potenciál"
  - "Áramerősség"
---
```

A `topics` mező opcionálisan bővíthető.

12. Markdown formátum

A Markdown legyen ember által is könnyen szerkeszthető.

Példa:

```
---
title: "Villamos alapmennyiségek"
subject: "Elektrotan"
date: "2026-09-03"
topics:
  - "Elektromos töltés"
  - "Feszültség"
  - "Áramerősség"
---

# Villamos alapmennyiségek
```

Elektromos töltés

A tananyag eredeti tartalma...

Elektromos mező

A tananyag eredeti tartalma...

Feszültség

A tananyag eredeti tartalma...

A Markdown legyen a tartalom elsődleges forrása.

13. Leckeoldal felépítése

Egy leckeoldal vizuális felépítése:

Breadcrumb

Elektrotan

2026. szeptember 3.

Villamos alapmennyiségek

Tartalom

[Markdownból generált tartalom]

Kapcsolódó témák

[Elektromos töltés]

[Feszültség]

[Áramerősség]

← Előző lecke

Következő lecke →

A tartalom legyen jól olvasható, megfelelő sorhosszal.

14. Tipográfia

A szöveg olvashatósága kiemelten fontos.

Legyen:

- megfelelő betűméret;
- kényelmes sorköz;
- korlátozott szövegszélesség;
- világos hierarchia;
- jól elkülönülő H1/H2/H3 címek;
- jól olvasható listák;
- megfelelő térköz a szakaszok között.

A hosszú szöveges tartalom ne töltse ki a teljes monitor szélességét.

Javasolt maximális olvasási szélesség:

kb. 700–850 px.

A navigáció és egyéb dashboard-elemek természetesen lehetnek szélesebbek.

15. Képletek és műszaki tartalom

A forrásanyag műszaki és matematikai képleteket is tartalmazhat.

A rendszernek ezért megfelelően kell kezelnie:

- matematikai képleteket;
- mértékegységeket;
- görög betűket;
- hatványokat;
- törteket;
- műszaki jelöléseket.

Ahol indokolt, használható MathJax vagy KaTeX.

Például:

$$U = R \cdot I$$

vagy LaTeX:

$$U = R \cdot I$$

A képletek legyenek mobilon is olvashatók.

16. Táblázatok

A Markdown táblázatokat a weboldal formázott, responsive táblázatként jelenítse meg.

Nagyobb táblázatok esetén mobilon legyen vízszintes görgetési lehetőség.

A táblázatok ne törjék szét az oldal elrendezését.

17. Kódblokkok

Ha a későbbiekben kód vagy technikai formátum kerül a jegyzetekbe, a Markdown code blockokat megfelelően kell megjeleníteni.

18. Kiemelések

A Markdownból létrehozható:

- információs blokk;
- figyelmeztetés;
- fontos megjegyzés;
- példa;
- definíció.

Ezek legyenek vizuálisan elkülönítve, de visszafogottan.

Nem cél a minden mondathoz tartozó színes doboz.

19. Témák és címkék

A lecke mellett legyenek témák.

Például:

```
Elektrotan
├─ Villamos alapelemek
│   └─ Elektromos töltés
│   └─ Elektromos mező
```

- └ Feszültség
- └ Áramerősség

A témák lehetnek cross-linkek.

Ha több leckében szerepel ugyanaz a téma, a témaoldal mutassa az összes kapcsolódó leckét.

Például:

Téma: Feszültség

Kapcsolódó leckék:

- Villamos alpmennyiségek
- ...
- ...

A témák rendszerét úgy kell kialakítani, hogy később automatikusan bővíthető legyen.

20. Keresőrendszer

A teljes tudástárban legyen keresés.

A kereső keressen legalább:

- lecke címében;
- tantárgyban;
- témákban;
- lecke szövegében.

A találati oldalon jelenjen meg:

- lecke címe;
- tantárgy;
- dátum;
- releváns rövid részlet;
- témák.

A keresés legyen gyors és kliensoldali, ha a statikus architektúra ezt lehetővé teszi.

Nem szükséges backend adatbázis.

A keresőnek később több száz vagy akár több ezer lecke esetén is használhatónak kell maradnia.

21. URL-struktúra

Az URL-ek legyenek emberileg olvashatók.

Példa:

```
/
/subjects/
/subjects/elektrotan/
/lessons/
/lessons/villamos-alapmennyisegek/
/topics/
/topics/feszultseg/
/search/
```

A pontos URL-struktúrát a választott site generatorhoz igazítsa a Codex.

Kerülendő:

```
/page?id=1847
/article.php?id=52
```

22. Navigáció a leckék között

A leckeoldal alján legyen:

← Előző lecke

Következő lecke →

A navigáció mindig az adott tantárgy időrendi struktúráját kövesse.

Ha nincs előző vagy következő lecke, az adott elem ne jelenjen meg hibás linkként.

23. Breadcrumb

A leckeoldalakon legyen breadcrumb.

Például:

```
Főoldal
→ Tantárgyak
```

- Elektrotan
- Villamos alapmennyiségek

Ez javítja a navigációt és a felhasználói tájékozódást.

24. Oldalsó navigáció

Desktop nézetben opcionálisan használható sticky oldalsó tartalomjegyzék.

Hosszabb lecke esetén:

Tartalom

- Elektromos töltés
- Ion
- Elektromos mező
- Feszültség
- Potenciál
- Áramerősség

Mobilon ez legyen összezsukható vagy rejtett.

25. Dark mode

A weboldal támogassa a sötét módot.

Legyen:

- light mode;
- dark mode;
- lehetőleg rendszerbeállításához igazodó alapértelmezett mód.

A felhasználó manuálisan is tudjon váltani.

A választás lehetőleg maradjon meg a böngészőben localStorage segítségével.

A dark mode ne csak a háttérszínt változtassa meg.

Ellenőrizni kell:

- szöveget;
- kártyákat;
- táblázatokat;
- kódblokkokat;
- képleteket;

- linkeket;
 - navbar-t;
 - keresőt;
 - form elemeket.
-

26. Színrendszer

A dizájn használjon kevés, következetes színt.

Javasolt:

- semleges háttér;
- világos tartalomfelület;
- egy fő accent szín;
- egy másodlagos szín szükség esetén;
- külön státuszszínek.

A színek legyenek akadémiai, nyugodt és professzionális jellegűek.

Ne legyen túl sok különböző élénk szín.

A kontraszt legyen megfelelő.

27. Ikonok

Használható egy egységes ikonrendszer.

Például Bootstrap Icons.

Az ikonok csak segítsék az információ értelmezését.

Ne legyen minden cím mellett kötelező emoji.

28. Képek és média

A rendszer legyen felkészítve képek kezelésére.

Javasolt struktúra:

```
assets/  
  images/  
  icons/
```

A Markdownból hivatkozott képek jelenjenek meg megfelelően.

A képek:

- legyenek responsive-ok;
- kapjanak megfelelő alt szöveget;
- ne okozzanak layout shiftet, amennyiben ez elkerülhető.

29. Forrásanyag integritása

Ez kiemelten fontos.

A weboldal készítésekor az AI **ne írja át önállóan a tananyagot tankönyvvé.**

A rendszer elsődleges feladata:

- strukturálás;
- formázás;
- rendszerezés;
- kereshetővé tétel;
- navigálhatóvá tétel.

Nem cél a tartalom önálló „oktatói” újraírása.

Az eredeti jegyzet jelentése maradjon meg.

Nyilvánvaló:

- gépelési hibák;
- formázási hibák;
- elírások;
- mértékegység-formázási hibák

javíthatók.

Bizonytalan vagy szakmailag kétséges állítást az AI ne módosítson önállóan úgy, hogy közben új információt talál ki.

Ha később fact-checking funkciót szeretnénk, az külön munkafolyamat legyen.

30. A jelenlegi forrásanyag

A projekt jelenlegi alapanyagaként használható:

Iskolai tanulmányok.docx

A már strukturált változat:

```
iskolai-tanulmanyok-tudastar.md
```

A jelenlegi tartalom többek között az alábbi területeket tartalmazza:

- Mechanika, Gépelemek;
- Elektrotechnika, elektronika, villamosságtan;
- Gépjármű-szerkezetan;
- Elektrotan.

A Codex ne feltételezze, hogy ezek a későbbiekben az egyetlen tantárgyak.

Az architektúrának új tantárgyak hozzáadásával is működnie kell.

31. Tartalomkönyvtár

Javasolt:

```
content/  
├─ mechanika-gepelemek/  
├─ elektrotechnika/  
├─ gepjarmu-szerkezetan/  
└─ elektrotan/
```

Egy tantárgyon belül:

```
content/  
└─ elektrotan/  
    ├─ 2026-09-03-villamos-alapmennyisegek.md  
    └─ ...
```

A fájlnevek legyenek:

- kisbetűsek;
- ékezet nélküliek;
- kötőjellel tagoltak;
- stabilak;
- URL-barátok.

32. Ne legyen duplikált adat

A tantárgy és lecke adatai ne legyenek több helyen kézzel karbantartva.

Például ne kelljen külön:

```
subjects.json
lessons.json
lesson.html
```

fájlokban ugyanazt az információt többször megadni, ha az automatikusan előállítható.

A Markdown front matter legyen az elsődleges metadata-forrás.

33. Automatikus generálás

A rendszer automatikusan generálja:

- tantárgylistát;
- tantárgyoldalakat;
- leckelistákat;
- leckeoldalakat;
- témalistát;
- témaoldalakat;
- statisztikákat;
- legutóbbi leckéket;
- előző/következő navigációt;
- keresési indexet.

Ez legyen az egyik legfontosabb fejlesztési cél.

34. Új lecke hozzáadásának folyamata

A későbbi munkafolyamat legyen egyszerű.

Ideális eset:

1. Új jegyzet elkészül
- ↓
2. Word / AI feldolgozás
- ↓
3. Markdown létrejön
- ↓
4. Markdown bekerül a content könyvtárba
- ↓
5. Build
- ↓
6. Weboldal frissül

A weboldal kódját ehhez ne kelljen módosítani.

35. Validáció

A build folyamat ellenőrizze a Markdown fájlokat.

Legalább:

- hiányzó cím;
- hiányzó tantárgy;
- hibás dátum;
- hibás metadata;
- duplikált slug;
- hibás belső link;
- nem létező kép;
- hibás témahivatkozás

legyen felismerhető.

A hibák legyenek érthetőek.

Például:

```
ERROR:
content/elektrotan/2026-09-04-example.md
```

```
Missing required field:
subject
```

36. Hibakezelés

A weboldal ne generáljon törött oldalakat hibás tartalom miatt.

Ha egy adat hiányzik:

- jelezze build közben;
- lehetőleg állítsa meg a buildet;
- adjon értelmezhető hibaüzenetet.

A Codex ne használjon csendes fallbackeket olyan esetben, amikor azok adatvesztést vagy hibás tartalmat eredményezhetnek.

37. Teljesítmény

A weboldal legyen gyors.

Kerülendő:

- szükségtelen JavaScript;
- túlméretezett képek;
- fölösleges frontend framework;
- indokolatlan külső dependency;
- nehéz animációk.

A statikus oldal előnyeit használjuk ki.

38. Accessibility

A weboldal legyen akadálymentesség szempontjából megfelelő.

Legalább:

- szemantikus HTML;
 - megfelelő heading-hierarchia;
 - billentyűzettel használható navigáció;
 - megfelelő kontraszt;
 - képekhez alt;
 - form mezőkhöz label;
 - fókuszállapotok;
 - ARIA csak szükség esetén.
-

39. SEO

Mivel statikus weboldal készül, legyen megfelelő alap SEO.

Minden leckeoldal kapjon:

- egyedi `<title>`;
- megfelelő meta description;
- canonical URL lehetőség;
- korrekt heading-hierarchiát;
- Open Graph metadata lehetőséget.

A SEO azonban ne menjen a tananyag olvashatóságának rovására.

40. 404 oldal

Legyen egy kulturált 404 oldal.

Például:

Az oldal nem található.

Lehet, hogy a keresett tananyag már másik helyre került.

[Vissza a főoldalra]

[Keresés]

41. Favicon és alap branding

Legyen:

- favicon;
- egyszerű weboldalnév;
- egységes brand;
- esetleg később saját logó.

A branding legyen visszafogott.

42. Későbbi bővíthetőség

Az architektúra később támogathassa:

- kedvencek;
- könyvjelzők;
- tanulási állapot;
- „elolvastam” jelölés;
- ismétlési rendszer;
- vizsgafelkészítő mód;
- random kérdések;
- flashcardok;
- jegyzetek;
- kapcsolódó fogalmak;
- saját kommentek;
- haladási statisztikák.

Ezeket jelenleg nem szükséges implementálni.

A kód azonban ne legyen úgy kialakítva, hogy később lehetetlen legyen hozzáadni őket.

43. Adatmodell

A minimális lecke modell:

```
Lesson
├─ id / slug
├─ title
├─ subject
├─ date
├─ topics[]
├─ content
└─ optional metadata
```

A subject modell:

```
Subject
├─ slug
├─ name
├─ description
└─ lessons[]
```

A topic modell:

```
Topic
├─ slug
├─ name
└─ lessons[]
```

A kapcsolatokat lehetőleg automatikusan a Markdown metadata alapján generálja a rendszer.

44. Kódminőség

A Codex által létrehozott kód legyen:

- tiszta;
- érthető;
- moduláris;
- dokumentált;
- következetes;
- újrahasznosítható.

Kerülendő az egyetlen hatalmas fájlba épített rendszer.

A komponensek legyenek logikusan elkülönítve.

45. Projektstruktúra

A végleges struktúra a választott build rendszerhez igazodhat, de legyen hasonló:

```
school-knowledge-base/  
├── content/  
│   ├── mechanika-gepelemek/  
│   ├── elektrotechnika/  
│   ├── gepjarmu-szerkezettan/  
│   └── elektrotan/  
├── src/  
│   ├── components/  
│   ├── layouts/  
│   ├── pages/  
│   ├── styles/  
│   └── scripts/  
├── public/  
│   ├── images/  
│   └── icons/  
├── scripts/  
├── package.json  
├── README.md  
└── ...
```

A Codex a választott technológiának megfelelően módosíthatja ezt.

46. Dokumentáció

A projekt gyökerében legyen README.

A README tartalmazza:

- projekt célját;
- technológiai stack-et;
- telepítés módját;
- fejlesztői indítást;
- build parancsot;
- új lecke hozzáadását;

- új tantárgy hozzáadását;
- Markdown formátumot;
- deploy folyamatot;
- gyakori hibákat.

A jövőbeli önmagam számára is legyen érthető.

47. Fejlesztési munkafolyamat Codexben

A Codex ne próbálja egyszerre megvalósítani az egész projektet vakon.

Javasolt fejlesztési sorrend:

Fázis 1 – Projekt alap

- projekt létrehozása;
- technológiai stack;
- build rendszer;
- Bootstrap;
- alap CSS;
- Markdown feldolgozás.

Fázis 2 – Design system

- színek;
- tipográfia;
- spacing;
- kártyák;
- gombok;
- navbar;
- breadcrumb;
- dark mode.

Fázis 3 – Tartalommodell

- front matter;
- lesson modell;
- subject modell;
- topic modell;
- slug rendszer.

Fázis 4 – Oldalak

- főoldal;
- tantárgylista;
- tantárgyoldal;
- leckeoldal;
- témák;

- témaoldal.

Fázis 5 – Keresés

- index generálása;
- kereső UI;
- találati oldal;
- keresési eredmények.

Fázis 6 – Navigáció

- breadcrumb;
- előző/következő lecke;
- kapcsolódó témák;
- tartalomjegyzék.

Fázis 7 – Minőség

- responsive;
- accessibility;
- SEO;
- 404;
- teljesítmény.

Fázis 8 – Tartalomimport

A meglévő Markdown tartalom integrálása.

Fázis 9 – Tesztelés

Az egész rendszer végigtesztelése.

48. Tesztelési követelmények

A Codex minden nagyobb fejlesztési fázis után ellenőrizze:

Funkcionális teszt

- működik-e a navigáció;
- működnek-e a linkek;
- működik-e a keresés;
- megjelennek-e a leckék;
- helyesek-e a tantárgyak;
- működnek-e a témák.

Responsive teszt

Legalább:

- mobil;
- tablet;
- desktop.

Tartalomteszt

Ellenőrizze:

- címeket;
- dátumokat;
- képleteket;
- táblázatokat;
- felsorolásokat;
- speciális karaktereket;
- magyar ékezeteket.

Dark mode

Ellenőrizze minden fő oldaltípuson.

49. Magyar nyelv

A teljes weboldal elsődleges nyelve magyar.

Fontos:

- magyar ékezetes karakterek;
- magyar szövegek;
- magyar dátumformátum;
- magyar navigáció;
- magyar UI-elemek.

A technikai slugok természetesen lehetnek ékezet nélküliek.

Például:

Villamos alapmennyiségek

de URL:

/villamos-alapmennyisegek/

50. Dátumok

A forrásanyag dátumai legyenek megőrizve.

Belső adatként:

2026-09-03

Megjelenítéskor:

2026. szeptember 3.

A dátum ne vesszen el az importálás során.

51. Tartalom sorrendje

A leckék alapértelmezett sorrendje dátum szerint legyen.

Ha azonos dátumon több lecke van, használjon stabil másodlagos rendezést.

A sorrend ne függjön véletlenszerű fájlrendszer-sorrendtől.

52. Importálási szabályok

A meglévő jegyzetek importálásakor:

- az eredeti tartalom maradjon meg;
- a szerkezet legyen javítva;
- a címek legyenek konzisztens formában;
- a nyilvánvaló elgépelések javíthatók;
- képletek legyenek megfelelően formázva;
- mértékegységek legyenek következetesek;
- bizonytalan tartalmi állításokat ne módosítson önállóan.

A forrásanyag hiányzó információit ne pótolja kitalált adatokkal.

53. Fontos tartalmi alapelv

Az AI itt elsősorban:

szerkesztő és rendszerező

és nem:

a jegyzet szerzője vagy tanára.

A tudástár célja a saját tananyag digitális rendszerezése.

54. Design részletek

A vizuális hierarchia legyen egyértelmű.

Fontos:

- sok whitespace;
- kerekített, de nem túlzottan lekerekített kártyák;
- finom border/shadow;
- megfelelő kontraszt;
- visszafogott hover állapotok;
- következetes spacing;
- stabil content width.

A kártyák ne legyenek túlzsúfoltak.

A tartalom legyen az oldal főszereplője.

55. Animációk

Csak finom animációk legyenek.

Például:

- hover;
- menu;
- dark mode;
- accordion.

Kerülendő:

- folyamatos mozgás;
- túlzott transition;
- dekorációs animációk;
- lassító effectek.

A weboldal tanulási eszköz, nem marketingoldal.

56. Biztonság

Mivel statikus oldalról van szó, ne legyen felesleges backend.

A Markdownból generált HTML-t megfelelően escape-elni kell.

A tartalomban szereplő HTML kezelését tudatosan kell megoldani.

A Markdown parser beállítása ne engedjen nem kívánt veszélyes tartalmat.

57. Külső függőségek

A lehető legkevesebb külső függőség használata javasolt.

A szükséges dependency-k legyenek indokoltak.

A Codex dokumentálja, hogy:

- mire való;
 - miért szükséges;
 - hogyan frissíthető.
-

58. Deployment

A végső rendszer legyen könnyen publikálható statikus hostingra.

Lehetséges platform például:

- GitHub Pages;
- Cloudflare Pages;
- Netlify;
- Vercel;
- más statikus hosting.

A konkrét platform később kiválasztható.

A build folyamat legyen automatizálható.

Ideális esetben:

```
Markdown módosítás
  ↓
Git commit
  ↓
```

```
push
  ↓
automatikus build
  ↓
weboldal frissül
```

59. Git használata

A projekt Git repositoryban legyen.

Ajánlott:

```
main
```

branch stabil verzióhoz.

Fejlesztés történhet külön branchben.

A commitok legyenek érthetőek.

60. Codex működési szabályok

A Codexnek a fejlesztés során:

1. Ne változtassa meg indokolatlanul a projekt architektúráját.
 2. Ne vezessen be új frameworköt megfelelő indok nélkül.
 3. Ne távolítsa el a Bootstrap 5-öt.
 4. Ne írja át a tartalmat saját tudása alapján.
 5. Ne találjon ki hiányzó tantárgyneveket vagy adatokat.
 6. Ne duplikálja a tartalmi adatokat.
 7. Ne hardcode-olja a leckéket a komponensekbe.
 8. A Markdown legyen a tartalom elsődleges forrása.
 9. Minden új funkció legyen kompatibilis a meglévő tartalommodellel.
 10. Minden nagyobb módosítás után futtasson buildet és tesztet.
 11. Hibás build esetén ne tekintse késznek a munkát.
 12. A kód legyen hosszú távon karbantartható.
 13. Magyar nyelvű tartalmat megfelelően kezeljen.
 14. A mobilnézetet minden UI-módosításnál vegye figyelembe.
 15. Ne generáljon felesleges funkciókat.
-

61. „Definition of Done”

A projekt akkor tekinthető első verzióban késznek, ha:

- [] a projekt buildelhető;
- [] a Markdown tartalom automatikusan feldolgozható;
- [] van működő főoldal;
- [] van tantárgylista;
- [] vannak tantárgyoldalak;
- [] vannak leckeoldalak;
- [] működik a témarendszer;
- [] működik a kereső;
- [] működik a breadcrumb;
- [] működik az előző/következő lecke navigáció;
- [] működik a responsive design;
- [] működik a dark mode;
- [] a képletek megfelelően jelennek meg;
- [] a magyar karakterek hibátlanul jelennek meg;
- [] nincs törött belső link;
- [] nincs hibás kép;
- [] van 404 oldal;
- [] van README;
- [] van validáció;
- [] a jelenlegi jegyzetek sikeresen importálhatók;
- [] új Markdown lecke hozzáadható kódmódosítás nélkül;
- [] a projekt statikus oldalként deployolható.

62. Első verzió prioritásai

Ha fejlesztés közben döntést kell hozni, a prioritás:

P0 – kötelező

1. Tartalomkezelés
2. Leckeoldalak
3. Tantárgyak
4. Navigáció
5. Keresés
6. Responsive design
7. Olvashatóság

P1 – nagyon fontos

1. Témák
2. Dark mode
3. Breadcrumb
4. Automatikus statisztikák
5. Validáció

- 6. SEO
- 7. Accessibility

P2 – későbbi fejlesztés

- 1. Kedvencek
- 2. Tanulási állapot
- 3. Ismétlés
- 4. Flashcard
- 5. Tesztek
- 6. Haladási statisztikák

63. A projekt filozófiája

A weboldal ne akarjon több lenni annál, amire szükség van.

A fő cél:

A saját iskolai tananyagom legyen egyetlen helyen, szépen rendszerezve, könnyen kereshetően és hosszú távon bővíthetően.

A rendszer legyen:

egyszerű használni → könnyű bővíteni → gyors → jól olvasható → stabil → szép.

A technológia szolgálja a tartalmat, ne fordítva.

64. Következő fejlesztési lépés

A jelen specifikáció alapján a következő feladat egy különálló, részletes **Codex 5.6 Terra master prompt** elkészítése.

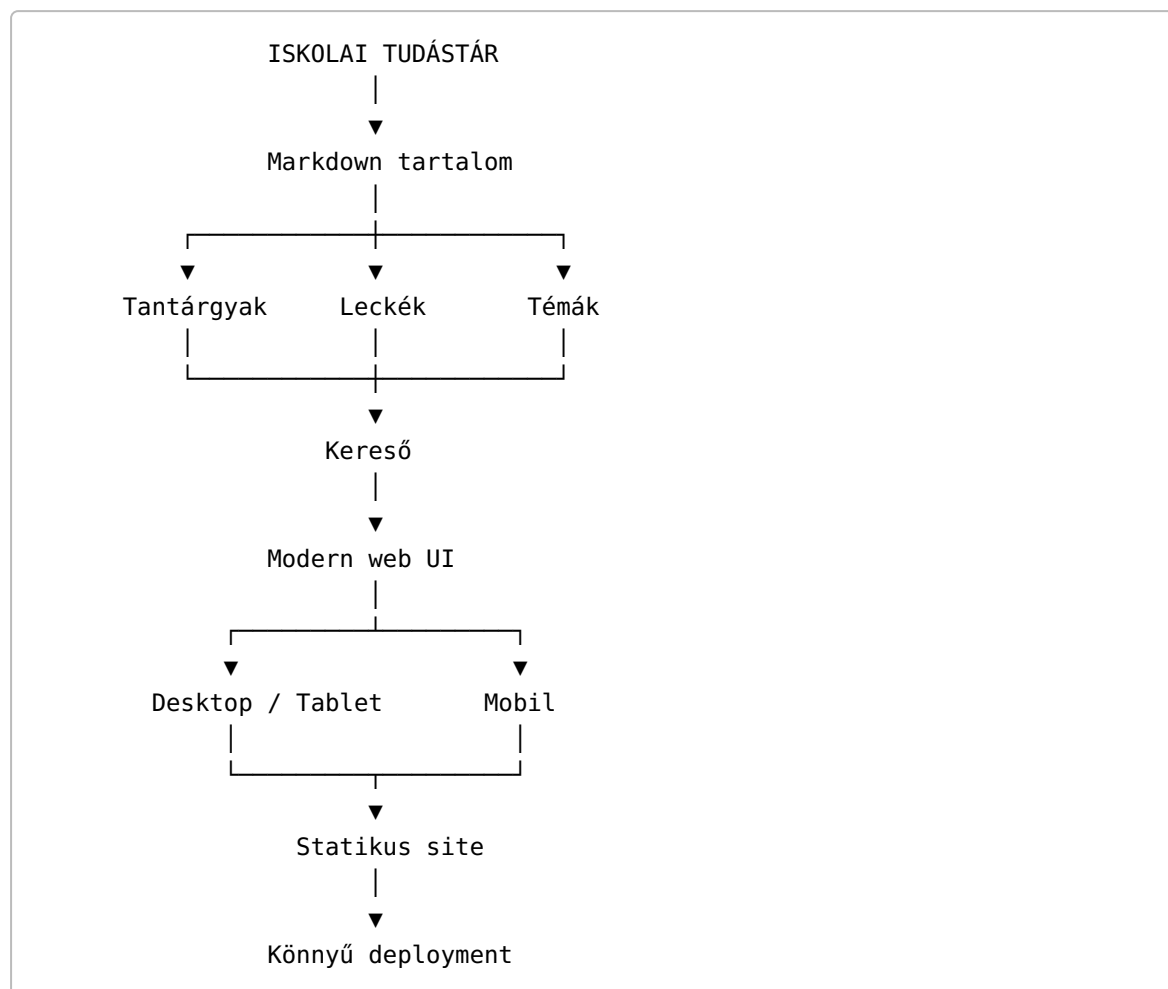
A master prompt feladata lesz:

- 1. a projekt inicializálása;
- 2. a technológiai környezet létrehozása;
- 3. a projektstruktúra kialakítása;
- 4. a Markdown tartalommodell implementálása;
- 5. a design system elkészítése;
- 6. a Bootstrap 5 integrációja;
- 7. a főoldal elkészítése;
- 8. a tantárgy- és leckeoldalak elkészítése;
- 9. a témarendszer elkészítése;
- 10. a kereső elkészítése;
- 11. a dark mode elkészítése;
- 12. a responsive működés biztosítása;

13. a meglévő tananyag importálása;
14. validáció és tesztelés;
15. README és fejlesztési dokumentáció létrehozása.

A Codex minden lépés után ellenőrizzé a működést, és csak működő állapot után folytassa a következő fázissal.

Rövid projekt-összefoglaló



Alapelv: a tartalom hosszú távon bővíthető, a weboldal pedig automatikusan alkalmazkodik az új leckékhez.